

Development documentation

In this section, we provide the development documentation of the Enhanced NKOD Dataset Search system.

Technologies

The majority of the application is written in Python. We use **Flask** as the web framework, the **pandas** package for handling tabular data, and the **llama_index** package for LLM-related tasks. Because the application has a web interface, it also uses HyperText Markup Language (HTML), Cascading Style Sheets (CSS) and JavaScript (JS) for web page creation, styling, and dynamic elements. Jinja is used as a templating engine for dynamic HTML generation and prompt templates.

The application also uses several software tools and systems, and the related Python packages for interacting with them.

Ollama

Ollama [?] is an open-source tool that allows users to download and run LLMs locally. It is used to run the LLMs and embedding models used by our system. For direct interaction with the Ollama server, the **ollama** Python package is used. The **llama_index** package also supports Ollama through the **llama_index.llms.ollama** subpackage.

Postgres

Postgres (also known as PostgreSQL) [?] is an open-source relational database system. We use Postgres as both our document store and vector store. Although there are native vector databases, such as Weaviate, Postgres with the **pgvector** extension can also function as a vector database. We decided to use Postgres because of its ease of use. For database management through Python, we use **PGVectorStore** and **PostgresDocumentStore** from **llama_index** and the **psycopg2** package.

Neo4j

As was already mentioned, Neo4j is used for the knowledge graph. For interacting with the graph database, we use the **neo4j** Python package, which allows us to execute Cypher queries.

LLM usage

LLMs are a crucial technology used in this work. They are used during data processing, in the search pipeline, and they power the chatbot.

Prompting

In the first chapter, we described the n-shot prompting technique. In our system, we use exclusively zero-shot prompting. To most of the prompts, we add a very short introduction to the problem domain:

```
You are an AI assistant for the National Czech Open Data  
Catalog - Národní katalog otevřených dat (NKOD).
```

Many prompts are designed to return structured output in JSON format. During implementation, we encountered problems with incorrect output formatting, so all prompts in this category contain extra instructions about the output format:

```
Return only the final JSON object. The whole output must be  
directly parseable by json.loads().
```

Generally, the prompts are designed to be as short as possible. The prompt files are Jinja2 templates, and some parts of the prompts are given to the LLM only if they are necessary or relevant.

LLMs and embedding models used

We explored several LLMs and embedding models.

For data processing LLMs, we tried gpt-oss-20b, Mistral-Small-3.2-24B-Instruct-2506 and Qwen3-14B. We decided to use the Mistral Small 3.2 model because it reliably generated the desired structured output in JSON format. With other models, we saw a greater number of formatting errors. Compared to the Qwen3-14B model, Mistral Small 3.2 was also significantly faster.

For the search platform, we compared Mistral Small 3.2 and Qwen3.6-35B-A3B. Here, Mistral Small 3.2 again had a significantly shorter response time. In the end, after evaluating both models, we used Mistral Small 3.2 for the search platform as well.

We use Qwen3-Embedding:4B as the embedding model in our system. We also tried snowflake-arctic-embed2 and Qwen3-Embedding:8B. The snowflake-arctic-embed2 model did not perform well. It has only 568 million parameters and an embedding size of 1024 dimensions, which may explain why it did not capture semantic similarity as effectively as the Qwen3 embedding models. The Qwen3-Embedding:8B model performed well, but we did not see a noticeable difference in results when compared to Qwen3-Embedding:4B. We decided to use Qwen3-Embedding:4B, because it is a smaller model and has a smaller embedding size, so it uses less memory, and the embedding generation is faster.

Project structure

The project contains the following directories:

- **data:** stores NKOD data used in the application.
- **documentation:** contains project documentation.
- **evaluation:** contains an evaluation script and a configuration file for evaluation.
- **llm_inputs:** contains LLM prompt templates and context files that are used by the LLM.
- **src:** source code directory.
- **tests:** contains unit tests and testing data.

EnhancedNkodDatasetSearch package

The Enhanced NKOD Dataset Search code is provided as a Python package called **EnhancedNkodDatasetSearch**. It contains two subpackages: **data_processing** and **search_platform**. The **data_processing** package contains the source code for data processing, and the **search_platform** package contains the source code for the search platform, including HTML templates, CSS, and JS files for the web application frontend.

Package structure

- **data_processing:** the data processing package.
- **search_platform:** the search platform package.
- **ollama_client.py:** definition of the `OllamaClient` class. This client is used for all explicit LLM calls, which include all LLM calls except those performed by the chatbot. It provides an interface with two methods: `get_llm_response` for unstructured output and `get_llm_json_response` for structured output.

- `utils.py`: module with helper functions, for example, for setting up a logger or for Jinja template rendering.

Modules `ollama_client.py` and `utils.py` are used by both subpackages.

Data processing package

The data processing package has the following structure:

- **NKOD**: directory containing NKOD-specific data processing code.
 - `kg_queries.py`: functions for executing Cypher queries over the knowledge graph.
 - `metadata.py`: contains functions used for metadata cleaning and enhancement during dataset processing.
 - `nkod_dataset_processing.py`: defines the `NkodDatasetProcessingPipeline`.
 - `nkod_documents.py`: defines `NkodDocumentConverter` that creates documents from processed datasets' metadata.
 - `nkod_knowledge_graph.py`: defines the `NkodKnowledgeGraph`.
 - `spatial_and_temporal_data_sparql.py`: contains functions for retrieval of spatial and temporal coverage from the NKOD SPARQL endpoint.
 - `utils.py`: a module with helper functions for dataset processing.
- `database.py`: module that handles communication with the Postgres database.
- `dataset_processing_pipeline`: defines the abstract `DatasetProcessingPipeline` class.
- `documents.py`: defines the abstract `DocumentConverter` class.
- `knowledge_graph.py`: defines the abstract `KnowledgeGraph` class.
- `main.py`: Python module that sets up and runs the data processing pipeline.

Postgres database management

Module `database.py` handles Postgres database management. It defines a class called `Database`, which contains a method `load_documents`. This method creates the staging vector and document tables, runs the ingestion pipeline to load the documents and embeddings to the staging tables, and promotes the staging tables to production once they are ready. It uses `PGVectorStore` and `PostgresDocumentStore` from `llama_index` to manage the vector and document stores. For direct manipulation with the Postgres database, which is needed for the promotion of staging tables to production, we use the `psycopg2` package.

During the database management implementation, an issue was identified in `llama_index`'s batch processing functionality, where embedding requests were processed sequentially instead of in batches. A correction was applied to the `llama_index.embeddings.ollama` package to enable proper batch execution, which resulted in significant reduction in processing time during embedding generation.

The `Database` class is also used by the search platform during retrieval. The retrieval step uses a vector retriever and a keyword-based retriever, which use the document store and vector index provided by the `Database` class.

Abstract classes

The data processing package defines three abstract classes: `DatasetProcessingPipeline`, `DocumentConverter`, and `KnowledgeGraph`. Each of these defines the interface that the data processing pipeline expects. Each of these classes should be specialized based on the used data catalog.

NKOD specializations

In the NKOD directory are specializations of the abstract classes for the dataset processing pipeline, knowledge graph, and document converter.

Dataset processing pipeline specialization, `NkodDatasetProcessingPipeline` handles the processing of the raw metadata dataset. Method `update_datasets` handles the daily update of datasets: it removes removed datasets, detects new or updated datasets, enhances their metadata, and adds them to the enhanced metadata dataset.

Method `get_new_datasets` compares the previous and current versions of metadata datasets to detect which datasets were removed and which are new or updated. It returns a pandas `DataFrame` of new or updated datasets and a list of URLs of removed datasets.

Method `enrich_new_datasets_metadata` handles the LLM-based enrichment of datasets' metadata. To speed up processing, the method is asynchronous, and it sends datasets for enrichment in batches.

Document converter specialization, `NkodDocumentConverter`, creates a `llama_index Document` from each dataset metadata. A document for each dataset is created, and the resulting list of documents is passed to the `load_documents` method from the `Database` class.

Knowledge graph specialization, `NkodKnowledgeGraph`, handles the creation and update of the knowledge graph. The `create_or_update_kg` method receives the enhanced metadata dataset, a `DataFrame` containing new or updated datasets, and a list of removed dataset URLs. If the number of new or updated datasets is the same as the number of datasets in the enhanced metadata dataset, the knowledge graph is recreated. Otherwise, it is updated.

The knowledge graph class is used by the search platform for the retrieval of similar datasets. Method `get_similar_datasets` handles the retrieval of similar datasets.

Data processing pipeline

The data processing pipeline is defined in the `main.py` module of the data processing package. It performs dataset processing, document creation, loads the created documents to the Postgres database, and updates the knowledge graph.

Search platform package

The search platform package has the following structure:

- `search_pipeline`: directory containing the search pipeline code.
 - `pipeline.py`: implementation of the search pipeline.
 - `query_preprocessing.py`: the query preprocessing module.
 - `retriever.py`: the retrieval module.
 - `result_postprocessing.py`: the result postprocessing module.
- `static`: directory containing static files used on the frontend – CSS styles, JS scripts and the application icon.
- `templates`: directory containing all HTML templates; one for each page of the application (home, search results, dataset detail, and chatbot), the footer template that defines the application footer and chatbot button template that creates the floating button that redirects the user to the chatbot page.
- `chatbot.py`: contains the chatbot backend logic.

- `data_catalog.py`: defines the abstract `DataCatalog` class.
- `nkod_data_catalog.py`: defines a specialization of the data catalog class: `NkodDataCatalog`.
- `main.py`: Python module that initializes and configures the Flask app and defines the routes.
- `utils.py`: Python module that contains functions used in the `main` module.

Search pipeline

As was already mentioned, the search pipeline has three steps: query preprocessing, retrieval, and search result postprocessing. Each step is implemented in a separate module.

The **query preprocessing** module defines a class called `QueryPreprocessor` that handles the preprocessing. It has a method `run` that calls `detect_user_intent` and `extend_user_query` in this order. Each step can be skipped, depending on the preprocessing configuration.

The **retrieval** module defines a class called `Retriever`. This class uses the vector index and document store from the `data_processing/database.py` module to retrieve relevant documents from the Postgres database. We use `QueryFusionRetriever` from `llama_index`, which can combine results of multiple retrievers and multiple queries.

The fusion retriever combines two retrievers: `BM25Retriever` and `VectorIndexRetriever`. The `BM25Retriever` performs keyword-based search in the document store using the BM25 algorithm. The `VectorIndexRetriever` performs vector search in the vector store based on the cosine similarity between embedding vectors. The results from these two retrievers are combined to obtain the final list of search results.

The `QueryFusionRetriever` has a parameter `num_queries`, which influences whether and how many additional search queries should be generated. If it is set to 2 or more, additional queries similar to the original one are generated, and the results for all queries are combined. We set this parameter to 1, so that no extra queries are generated, because we already generate an extended query in the preprocessing step.

Another important parameter of the `QueryFusionRetriever` is `retriever_weights`, which specifies the influence of each of the combined retrievers. By default, both retrievers have the same influence, but we can set a larger weight for one, so that it has a greater influence on the final list of results.

We pass a single query to the fusion retriever: a combination of the original search query, the extended query, and a list of all applied metadata filters. We tested multiple approaches, such as running the retrieval multiple times and then combining the results: for the original query, the extended query, and the original query combined with metadata filters. However, we observed the best results when using just one combined query.

The **result postprocessing** module defines a class called `PostProcessor`, which handles the postprocessing of the retrieved results. It receives a list of retrieved results together with their retrieval score. It reranks the results using the specified reranking approach, removes results with a retrieval score below the specified threshold, and keeps at most 30 of the remaining results.

Reranking approaches were already described; the retrieval score-based approach is called `simple`, and the cross-encoder-based approach is called `cross_encoder`. For cross-encoder reranking, we use the *mmarco-mMiniLMv2-L12-H384-v1* multilingual cross-encoder trained on the MS MARCO[?] dataset, machine translated into multiple languages. Unfortunately, Czech is not included in the languages, but we did not find any open-source cross-encoder that was trained on Czech data.

It can be configured which reranking approach should be used. We observed significantly better results with the `simple` approach. This approach is also faster, because it uses already computed scores.

Module `pipeline.py` then implements the whole search pipeline as a class `SearchPipeline`. It receives a search query as input, performs the defined steps, and returns a list of search results.

Chatbot

The chatbot is implemented as a class called `Chatbot`. It uses `FunctionAgent` from `llama_index`, an agent with function-calling capabilities. We provide four tools for the agent:

- `search_in_data_catalog`: uses the search pipeline to search for datasets in the catalog. The agent constructs a search query based on the previous conversation.
- `get_list_of_all_values_for_metadata_category`: returns a list of all values for one of the following metadata types: themes, categories, and provider.
- `get_data_catalog_information`: reads a file with information about the NKOD or the Enhanced NKOD Dataset Search system.
- `get_dataset_info`: returns dataset metadata based on its title or URL.

The agent decides what tools to use, invokes them, and constructs an answer based on the results. The `react_to_message` method runs in a loop and processes each received user message.

Data catalog representation

The search platform needs access to the data catalog. For that, a specialization of the `DataCatalog` class is needed. We implement the `NkodDataCatalog` specialization. This class provides an interface for getting dataset metadata. It reads the enhanced metadata dataset file and provides a method `get_dataset_by_url`, which returns the complete dataset metadata as a dictionary. It also instantiates the `NkodKnowledgeGraph` class and provides a `get_similar_datasets` method, which retrieves similar datasets from the knowledge graph.

Web application

The `main.py` module defines all routes for the web application. The `home`, `search_results`, `dataset_detail`, and `chatbot` routes handle the respective web page rendering. The `reset_chatbot` route handles resetting the chatbot memory, `chatbot_query` passes the user message to the chatbot backend and `set_language` sets the chatbot language.

Session management is implemented using Flask's `session` object. For each session, new `SearchPipeline` and `Chatbot` objects are created. These objects are stored on the server side in global dictionaries.

Objects shared between sessions (`DataCatalog`, `Database`, and `KnowledgeGraph`) are initialized during the Flask application initialization.

Installation and run

In this section, we explain how to install the `EnhancedNkodDatasetSearch` package and how to run it locally.

Software dependencies

To install and run the provided package, a Linux operating system or a WSL2 environment is needed. The installation requires Python 3.12 or higher, Ollama, PostgreSQL, and Neo4j. The system was developed using Python 3.12.13, PostgreSQL 14.23, and Neo4j version 2026.05.0. A remote server running Ollama 0.21.0 was used.

The system uses LLMs through Ollama. Before running the system, the required LLM and embedding model must be downloaded. To download the required models, run:

```
ollama pull mistral-small3.2:latest
ollama pull qwen3-embedding:4b
```

To run both of these models with the default configuration, approximately 37 GB of free memory is needed. In case of insufficient memory, an LLM and an embedding model that require less memory can be downloaded, and the system can be configured to use them instead. System configuration is explained later in this chapter.

Installation instructions

The `EnhancedNkodDatasetSearch` package can be installed by running the following command from the project root folder:

```
pip install -e .
```

This will install the package together with all of the required Python packages specified in `pyproject.toml`.

Local run

Creating a `.env` file

To run the data processing or the web application, a `.env` file with the following variables is needed:

```
POSTGRES_USER=<postgres-user>
POSTGRES_PASSWORD=<postgres-password>
POSTGRES_DB=<database-name>
NEO4J_USER=<neo4j-user>
NEO4J_PASSWORD=<neo4j-password>
NEO4J_URI=<neo4j-uri>
SECRET_KEY=<flask-secret-key>
CONFIG_PATH=config.yaml
```

These variables are used for connection to the Postgres and Neo4j databases, `SECRET_KEY` is used for the encryption of Flask user session data, and `CONFIG_PATH` specifies the path to the configuration file that is used by the web application.

Postgres database

If the Postgres database specified in the `POSTGRES_DB` variable does not exist, it must be created.

Running data processing

Data processing can be started by running the following command from the project root folder:

```
python -m EnhancedNkodDatasetSearch.data_processing.main
--config_path config.yaml
```

This will start the data processing pipeline.

Running the web application

The web application can be started by running the following command from the project root folder:

```
flask --app EnhancedNkodDatasetSearch.search_platform.main run
```

Configuration

All system configuration is handled through a configuration file `config.yaml`. This file allows us to specify the data paths, database configuration, search platform parameters, and data processing parameters without modifying the source code. It is organized into the following sections:

- **data:** configuration of the download URLs for the raw metadata dataset and the distribution dataset, and where the original and processed files should be stored.
- **state_dir:** specifies the directory that stores application state.
- **app_url:** the URL of the provided search platform application.
- **db:** Postgres database configuration. Here, the database host and port are specified, together with the document table name, vector table name, and the embedding dimension. The embedding dimension must match the embedding dimension of the embedding model used.
- **embedding:** the embedding model configuration. Here, the embedding model name and URL are specified. **chunk_size** specifies the maximum token length of a chunk. If a dataset document is longer, it is split into multiple chunks, and each is embedded separately. **chunk_overlap** specifies the token overlap between these documents.
embed_batch_size specifies the number of documents that are sent together for embedding.
url_to_node_id_mapping_file is a file that stores the mapping of dataset URLs to their vector database node IDs.
- **search_platform:** the search platform configuration.
 - **llm:** the LLM configuration for the search platform, consisting of the model name, URL, context length, and memory limit for the chatbot.
 - **preprocessing:** the query preprocessing configuration, consisting of whether intent detection and query extension should be performed, and the timeout in seconds for the LLM requests.
 - **retriever:** the retrieval configuration, consisting of the number of retrieved results from each of the combined retrievers and from the fusion retriever, the fusion mode, and the retriever weights.
 - **postprocessing:** the search results postprocessing configuration, consisting of the reranking approach, the cross-encoder model used in case we use cross-encoder reranking, the retrieval score threshold, and the maximum number of search results returned.
- **data_processing:** the data processing configuration.
 - **llm:** the LLM configuration for data processing, consisting of the model name, URL, context length, and timeout in seconds.
 - **knowledge_graph:** knowledge graph configuration, consisting of the maximum number of similar datasets retrieved, the knowledge graph name, and the semantic similarity threshold. The threshold specifies the minimum semantic similarity score needed in order for two datasets' documents to be considered semantically similar.
 - **column_mapping:** column mapping for the raw metadata dataset. Specifies what each column should be renamed to.
 - **distribution_column_mapping:** column mapping for the distribution dataset.
 - **irrelevant_columns:** list of columns that should be dropped from the raw metadata dataset.

- **categories**: list of dataset categories. For our system, this list is based on the Data theme vocabulary.
- **other_category**: an additional category that is used when the LLM cannot classify a dataset into any of the defined categories.

To change the configuration of a specific part of the provided system, the corresponding section of the configuration file must be updated. No changes to the source code are required.